

How to Use AI Without Letting It Think for You

10 min read · Jul 13, 2025



Simon Thornewill von Essen

Following ▾

Summary (TL;DR):

- Large Language Models (LLMs) aren't magic. Used lazily, they'll degrade thinking and outputs. When used well, they'll do the opposite.
- This post outlines four principles for making that happen:
 1. *Staying in the driver's seat.*
 2. *Step back when you can't confirm the output.*
 3. *Use LLMs for narrowly defined tasks.*
 4. *Have the human provide the scaffolding to direct those tasks.*
- The payoff for effective LLM use isn't just speed but better thinking, which is useful everywhere.
- We will only get these benefits if we train people to use them well. If we do, then these tools can be key in helping us learn and perform at our best.

. . .

The advent of recent powerful Large Language Models (LLMs) and Generative AI (Gen AI) systems, such as ChatGPT, Gemini, Claude, Deepseek, and others, has created a plethora of powerful tools for us to utilise.

As with most technology, powerful new tools come with upsides and downsides. These tools can serve as powerful assistants, but are usually tasked with replacing humans which had led to many companies firing some portion of their developers hoping that the efficiency-gains from these chatbots will be enough to compensate, only to find that this wasn't the case and needing to re-hire to fill the gap left behind.

When LLMs are used to replace human work, the result is often substandard and is given the online nickname "slop". There's also concern that these tools make writing worse and make learning shallow, as can be seen from recent research from MIT.

Chatbots have also been integrated into coding environments like VS Code, creating Cursor, which utilises LLMs like Claude to automate some amount of code writing. This has given rise to what is known as "vibe coding," where you use human language to shape code with minimal understanding of the actual implementation logic of the generated code.

Unfortunately, despite the promise, recent research suggests that tools like Cursor may even lead to a decrease in productivity.

These studies suggest that LLMs are most detrimental when humans delegate entire tasks to them without exercising meaningful oversight and supervision.

For example, the participants in an MIT study were required to use GPT-4o as their only information source for a writing task — effectively mimicking the worst imaginable use case: complete dependency. Unsurprisingly, their output and understanding suffered as a result.

A more realistic study of developers using Cursor revealed similar limitations: while LLMs aided with simple tasks, performance declined in complex, mature codebases where strategic thinking and precision are most crucial.

Both studies reinforce the same principle: In order to make the most of these tools, we need to stay engaged. In this post, I want to outline the principles that will allow you to do just that.

AI use in the Chess and Go world provides an example of what good AI use looks like

In both Chess and Go — complex games of strategy, skill, creativity and calculation— AI has been able to play at superhuman levels since 1997 and 2016, respectively. Two important steps are widely known when using these tools to learn from your games:

1. Conduct an initial review on your own and use these tools to check your work. The harder you work at reviewing the game on your own, the more you will learn from the AI review.
2. Spend time exploring the branches of the AI's analysis. Since they have super-human positional and reading skills, you should never accept their conclusions at face value. If there is a move that seems surprising or confusing, you need to employ Socratic questioning to understand why your suggestions are not what it chooses to play.

Failing to do both steps will mean that you will learn minimally from it and, in the worst case, you will start mimicking it without the super-human skills to back it up, which can temporarily make your play worse.

However, performing these two steps means that every player has access to a super-human level coach at all times, even if this coach cannot speak. It can help beginners spot obvious blunders and, for more advanced players, progress their understanding of the game.

Four principles of LLM use

1. Stay in the Driver's Seat! The co-pilot must truly remain as such

As a human, you must be able to have as complete oversight as possible of both the project you are working on and the output that these tools generate.

LLMs are helpful, but don't have the context of a wider project, the use case, and are fundamentally just predicting the next token. They mimic human thought, but are not (and may never, given the current architecture, be) an actual replacement for it.

Humans employing an LLM have the role of providing that extra step in logical thinking and broader context that the machine will lack, in my opinion, until we have superintelligent AI. Until then, when using it, our jobs shift towards oversight, strategy and confirmation.

If we struggle to do this, then the efficacy of what we produce together falls. The output becomes convoluted and will struggle to add value.

2. Struggling to confirm the output means you should step back

The implication of the previous point means that a key risk of using LLMs is that, as a human, you will tire where the machine will not, and it will not tell you when you should take a break.

Your goal is to maintain your discipline, which means respecting your boundaries as a living organism. As your mind tires, you will outsource more of your thinking to the machine, and this is when things will go wrong.

Luckily, things don't go wrong silently. You will struggle to confirm and understand the output either because you don't fully understand the code or because the LLM is using concepts that you don't understand. This leads to blindly trusting its output.

For example, suppose you are using LLMs to learn about statistical techniques and concepts. In that case, you must investigate every concept that's important for your use case, similar to how you would if using an AI to review a game of Chess or Go.

The longer you delay, the more you will need to step back and do the work that you should have been doing all along. This can mean a dedicated research or refactoring session. If you do your due diligence then hopefully, with practice, the

frequency of these sessions will decrease.

3. Use the AI for narrowly defined sub-tasks rather than the whole project

It's clear that if you have an LLM develop an entire essay from scratch with the human barely giving their input, as is what happened with the MIT study, then you can expect more shallow learning and a lacklustre output.

In writing this very post, I used LLMs as thought partners to bounce around ideas, suggest structure, and generate feedback. Each suggestion was processed, edited, or rejected by me. (Often saved in my Obsidian vault in the form of hand-written notes.) This back-and-forth preserved the integrity of my thinking while leveraging their power.

If I were to ask an LLM to generate this post for me, I would have skipped many important steps in attempting to solve this problem by myself, thinking about it from multiple angles, reading and noting the work of other people, and comparing and contrasting ideas before synthesising them all together into a first draft.

I'm still doing the work, and I'm not expecting LLMs to replace that work, but to help me do it better so that I can go further.

4. Start with human-defined scaffolding

If you are using an LLM to learn about a new topic or to write some code, you should lay the groundwork. This means having a baseline understanding of the area you are learning about, or by prompting it using TODO lists, pseudocode, examples, and providing context (such as preferred tools, etc.).

The more you're able to do this, to the extent that you are indeed using the AI for a narrow task, the better it will perform. In a sense, we also perform better when we focus on one task at a time, which means that when the AI focuses on what it does well, we are also able to concentrate on what we do best in turn.

Remember, even before LLMs became popular, websites like Stack Overflow often gave developers “enough rope to trip themselves”. We've already been encouraged to become more thoughtful in our approaches, and being able to utilise the resources we've access to to solve the right problem in the right way, while understanding the code being adapted, was already a key skill.

This means that as a human, you set the context, direction, and goals, and the better you can do this work, the better things will go.

LLMs are most powerful as learning tools when they are used to enable top-down, task-first learning

I'm not just advocating the use of LLMs for coding, but also for learning. This may make many readers uneasy, but writers such as Sal Khan of [Khan Academy](#) and author of the book “[Brave New Words](#)” also believe that LLMs will become a kind of tutor for students, similar to how we all have super-human Go/Chess coaches in our pockets, even if current students struggle to use them now.

Still, the unease of this proposition is somewhat justified. If an LLM gives you a misleading explanation or if you misunderstand its output, there can be consequences. I'm not advocating that someone uses AI to go straight into open-heart surgery on their first attempt, but if it could be used to help surgeons learn this skill more efficiently, then that would be beneficial.

The crux of the matter is that people don't develop mastery by waiting until they're “ready.” They learn by engaging with tasks just beyond their comfort zone, often in coordination with others, and ideally in environments where there is an appropriate amount of risk such that mistakes can be caught and reflected on.

This is exactly what the *zone of proximal development* describes, and when used wisely, LLMs can help guide learners within that space.

In my experience, applying Bayesian statistics and probabilistic programming with LLMs has helped me bridge the gap between theory and practice. For instance, I've used ChatGPT to help me implement an ordinal logistic regression in Bambi and then to create a working implementation in PyMC as well. This led me to wonder how to structure a hierarchical model within the same framework, and I implemented it after reading through the documentation and using

ChatGPT to clarify concepts that I didn't understand or address other questions I had. (The syntax used in the formula part of Bambi models.)

I was able to relate everything to what I learned from reading *Statistical Rethinking* and adapted it for my case. The important part is that I already possess many skills in statistics, which means I was able to verify the output. For instance, my PyMC code generates the same outputs as the Bambi package. I didn't go from knowing nothing about Bayesian modelling to trying to implement a hierarchical model in a generalised linear model. Instead, learners should focus on small objectives that immediately fall outside their comfort zone.

That's the essence of how these tools should be used. If you stay involved, clarify what you don't understand, and apply what you're learning to concrete problems, LLMs become study partners. Like a fellow student, it may know many things while having its gaps and misconceptions, but like a good study group, they can challenge you to reach beyond your current level if you remain an active participant.

Conclusion: It's Not the Tool, It's How You Use It

Throughout this blog post, we've considered how LLM use can both help and hinder towards the aims of learning, writing and coding. We've seen how these tools present a significant risk that can slow down development and impede cognitive function. These risks are real, but that does not mean they are inevitable.

However, these tools are new and come with a learning curve that risks short-term productivity loss and potentially risky real-world decisions, especially when they are used to replace human thought rather than augment and challenge it. Towards this end, I've developed the four principles that I've discussed above:

1. Stay in the driver's seat.
2. If you begin to lose understanding of an LLM's output, take a break and do the work to understand the output thoroughly.
3. Use LLMs for narrowly defined tasks, alternating input/outputs between the human and the machine.
4. Have the human provide as much scaffolding as possible to maximise the 3rd point.

Doing this means that the tools can enhance human learning rather than get in the way. To use these tools requires discipline and effort; those who seek to relieve humans of their work or avoid it will be disappointed.

Yet, I believe that I have demonstrated that there is also an immense opportunity in terms of creativity and productivity that lies ahead for those who take the time to understand and utilise these tools effectively. I hope that the concepts I have outlined here help both of us to achieve our own goals.
